

command to your directory. We can check our module by issuing the following commands:

```
aasisvinayak@GNU-BOX:~$ sudo dmesg -c > /dev/null
aasisvinayak@GNU-BOX:~$ sudo insmod module1.ko
aasisvinayak@GNU-BOX:~$ sudo dmesg -c
[74091.826941] Let's begin our new segment!
aasisvinayak@GNU-BOX:~$ sudo rmmod module1
aasisvinayak@GNU-BOX:~$ sudo dmesg -c
[74119.021887] Adieu to segment 2!
```

You can see that a *module1.mod.c* file was also created in the process:

```
#include <linux/module.h>
#include <linux/vermagic.h>
#include <linux/compiler.h>

MODULE_INFO(vermagic, VERMAGIC_STRING);

struct module __this_module
__attribute__((section(".gnu.linkonce.this_module"))) = {
    .name = KBUILD_MODNAME,
    .init = init_module,
#ifdef CONFIG_MODULE_UNLOAD
    .exit = cleanup_module,
#endif
    .arch = MODULE_ARCH_INIT,
};

static const struct modversion_info ____versions[]
__used
__attribute__((section("__versions"))) = {
    { 0xa257c5a3, "struct_module" },
    { 0xb72397d5, "printk" },
    { 0xb4390f9a, "mcount" },
};

static const char __module_depends[]
__used
__attribute__((section(".modinfo"))) =
"depends=";

MODULE_INFO(srcversion, "4675BE4AD96DEF402B04BD1");
```

You will find that the code requires the following files as well:

```
#include <linux/list.h>
#include <linux/stat.h>
#include <linux/compiler.h>
#include <linux/cache.h>
#include <linux/kmod.h>
#include <linux/elf.h>
#include <linux/stringify.h>
#include <linux/kobject.h>
```

```
#include <linux/moduleparam.h>
#include <linux/marker.h>
#include <asm/local.h>
#include <asm/module.h>
```

If you are an experienced programmer, then by looking at the dependencies you will know how the new file has been created. Novice users need not worry, as this column will address everything from scratch. For the time being, you can just explore the directories */lib/modules/2.x.x-version* and */usr/src/linux-headers-2.x.x-version*.

As I mentioned before, you can load a module by issuing commands. *lsmod* will list the modules (with a small description about its current usage) in your terminal. Figure 1 shows the result of the execution.

You can also view the list of modules to load at boot time by checking the */etc/modules* file. It will have entries similar to the one listed below:

```
# /etc/modules: kernel modules to load at boot time.
#
# This file contains the names of kernel modules that should be loaded
# at boot time, one per line.

fuse
lp
sbp2

# Generated by sensors-detect on Wed Jan 14 21:14:15 2009
# Chip drivers
coretemp
```

You can issue *modprobe* for probing. Its usage is shown below:

```
Usage: modprobe [-v] [-V] [-C config-file] [-n] [-i] [-q] [-Q] [-b] [-o
<modname>] [ -dump-modversions ] <modname> [parameters...]
modprobe -r [-n] [-i] [-V] <modulename> ...
modprobe -l -t <dirname> [-a <modulename>] ...]
```

As this is an introductory column, the kernel build process (which has five parts, viz. Makefile, .config, arch/\$(ARCH)/Makefile, scripts/Makefile, Kbuild Makefiles) will be explained in detail in subsequent articles.

Compiling the kernel

Now we will look at how to compile a kernel:

- Download the latest Linux source. If you have installed Linux with the kernel source option, the source will already be there. You may copy the kernel source to a safe location so that you can restore it later. It is a good idea to backup your */boot* directory as well or at least the kernel image. On my machine, the source was located in */usr/src/linux-2.6.27-7*.
- Create a soft link to your Linux folder, so that you